UrbanLens — Milestone 3 Report

Forecasting Pipeline & Model Evaluation

> IMPORTANT
>
> All test suites & production builds are passing cleanly on actual hardware:
>
> - **Python tests:** 51 / 51 passing (25 existing M2 tests + 26 new forecasting tests) ✅
> - **Server API tests:** 17 / 17 passing (10 existing M2 tests + 7 new forecasting endpoint tests) ✅
> - **Python CLI smoke test:** `run_forecast.py` returns valid JSON with MAE/RMSE/intervals ✅
> - **Client production build:** `tsc -b && vite build` exited with code 0 (2.04s) ✅

1. Git Checkpoint

Commit	Message
`bafe36e`	`feat: milestone 3 — forecasting pipeline, baselines, /api/forecasting/predict, Forecasting page integration`

2. Phase 1 Verification: Real vs. Synthetic Training Data Audit

Before training any forecasting models, the local data repository and manifests were audited:

- **`server/python/data/processed/processed_hpi.csv`** contains 13 observations across Mumbai (8 quarterly periods, 2020-01 to 2021-10) and Delhi (5 quarterly periods, 2020-01 to 2021-01).
- **All 13 records have `data_status: "synthetic_test"`.** There are **0 records** with `data_status: "real"`.
- **Sufficiency Analysis**: 8 quarterly observations are mathematically insufficient to fit seasonal time-series models like SARIMA (which require at least 2 full annual cycles: 24+ observations).

- **Core Governance Rule Enforced**:
  - The pipeline **strictly rejects** using `synthetic_test` fixture data for real-market forecast requests.
  - When querying `/api/forecasting/predict?mode=real`, the API returns an explicit `status: "unavailable"` with the human-readable explanation: *"No real-data observations available for geography 'Mumbai' and metric 'hpi_assessment'. Only synthetic test fixtures exist."*
  - It **never silently substitutes** synthetic observations for real data.

3. Phase 2: Python Forecasting Pipeline (`server/python/forecasting/`)

A reproducible, dependency-light forecasting module was built using only NumPy and Python standard library:

1. Data Sufficiency Checker (`data_check.py`)

- Defines minimum observation thresholds per model class (`naive_last: 1`, `seasonal_naive: 4`, `linear_trend: 4`, `sarima: 24`).
- Assesses input records for `has_real_data` and returns eligible/ineligible model lists and blocker messages.

2. Statistical Baseline Models (`baselines.py`)

- **`naive_last_value`**: Repeats the most recent observation forward for hh steps. Computes in-sample 1-step-ahead residuals.
- **`seasonal_naive`**: Extrapolates historical seasonal patterns ($y_{t+h} = y_{t+h-m}$ yt+h = yt+h−m where $m = 4$m = 4 for quarters or $m = 12$m = 12 for months). Provides residual standard-deviation prediction intervals.
- **`linear_trend`**: Ordinary Least Squares (OLS) regression over the time index with analytical 95%95% prediction intervals ($\pm 1.96 \cdot s_e \sqrt{1 + 1/n + (x - \bar{x})^2 / \sum (x_i - \bar{x})^2}$ ±1.96 · se 1 + 1/n + (x − x¯)2/∑(xi − x¯)2

).

3. Evaluation Metrics (`metrics.py`)

- **MAE & RMSE**: Exact mathematical error formulations.

- **MAPE**: Calculated safely with zero-division protection (returns `None` when actual values include zero; explicitly documented as suitable only for ratio-scale data, not index deviations).
- **Baseline Comparison**: `compare_to_baseline()` reports percentage error reduction over the `naive_last_value` baseline.

4. Engine & Chronological Splitting (`engine.py` & `demo_engine.py`)

- Employs strict **chronological 80% train / 20% validation splitting** with zero future-data leakage (verified by unit test `test_chronological_split_no_leakage`).
- **`demo_engine.py`**: Replicates the exact deterministic 60-month monthly price generation algorithm from `syntheticData.ts` for all 5 sectors (Jan 2020 – Dec 2024), clearly tagged with `data_status: "synthetic_demo"`.

4. Phase 3: Express API Endpoint

`GET /api/forecasting/predict`

Located in and mounted under `/api/forecasting`.

Query Parameters:

- `geography` (string, max 60 chars, regex-sanitized against path traversal)
- `metric` (enum: `hpi_assessment` | `hpi_market`, default: `hpi_assessment`)
- `horizon` (integer, bounded $1 \le h \le 24$1 ≤ h ≤ 24, default: 12)
- `mode` (enum: `demo` | `real`, default: `demo`)

Response Schema:

```
json
{
  "status": "ok",
  "geography": "mumbai-central",
  "metric": "price_per_sqft",
  "unit": "INR_per_sqft",
  "geography_level": "neighborhood",
  "data_status": "synthetic_demo",
  "source_names": ["SYNTHETIC_DEMO"],
  "training_period": { "start": "2020-01-01", "end": "2023-12-01" },
  "validation_period": { "start": "2024-01-01", "end": "2024-12-01" },
  "n_training": 48,
  "n_validation": 12,
  "model_name": "linear_trend",
  "model_version": "1.0",
  "horizon": 6,
  "forecast_dates": ["2025-01-01", "2025-02-01", "2025-03-01", "2025-04-01", "2025-
05-01", "2025-06-01"],
  "point_forecasts": [34975.56, 35180.75, 35385.94, 35591.13, 35796.32, 36001.51],
  "lower_bounds": [33485.14, 33690.33, 33895.52, 34100.71, 34305.90, 34511.09],
  "upper_bounds": [36465.98, 36671.17, 36876.36, 37081.55, 37286.74, 37491.93],
  "observations": [...],
  "forecast": [...],
  "evaluation": {
    "model": { "MAE": 901.67, "RMSE": 1089.87 },
    "baseline_naive_last": { "MAE": 109.50, "RMSE": 227.89 },
    "vs_baseline": { "MAE_reduction_pct": -723.45, "RMSE_reduction_pct": -378.24 }
  },
  "warnings": ["THIS IS A SYNTHETIC DEMO. Output does not represent real market
data."],
  "limitations": [
    "Model fitted on synthetic mathematical series for interface illustration.",
    "Under no circumstances should this be used for actual capital allocation,
appraisal, or lending decisions."
  ],
  "generated_at": "2026-10-29T10:37:08.951058+00:00"
}
```

5. Phase 4: Frontend Integration (`Forecasting.tsx`)

The UI was connected to the live `/api/forecasting/predict` endpoint:

- **Live API Hook (`useForecast`)**: Retrieves forecasts dynamically based on selected sector, horizon (3m, 6m, 12m), and mode.
- **Demo vs. Real Data Toggle**: Allows switching between demo simulation and real HPI forecasting. In `real` mode, clearly surfaces the graceful `EmptyState` explaining that real HPI series have not yet been imported.
- **Model Evaluation Card**: Displays the live model benchmark table:
  - Model Architecture (`linear_trend v1.0`)
  - Training Period (48 samples) & Validation Period (12 samples)
  - Validation MAE & RMSE with comparison against Naive Last Value baseline
- **Data Governance Badges**: Prominently renders `DataStatusBadge` (`SYNTHETIC DEMO DATA` / `DATA UNAVAILABLE`).
- **Interactive ComposedChart**: Renders historical rate line (dark slate), dashed forecast point estimate (indigo), and shaded 95% confidence intervals.

6. Test Suite Details

Python Tests (51 / 51 Passing)

```
Ran 51 tests in 0.066s — OK
- TestBaselines: 6 passed
- TestDataCheck: 5 passed
- TestDemoEngine: 4 passed
- TestEngine: 6 passed
- TestMetrics: 5 passed
- TestIngestion: 11 passed
- TestQuality: 1 passed
- TestValidators: 13 passed
```

Express Server Tests (17 / 17 Passing)

```
✔ 1. GET /api/health → 200, status: ok
✔ 2. GET /api/data/sources → 200, array of DataSource
✔ 3. GET /api/data/series → 200 (array of observations)
✔ 4. GET /api/data/series?start_date=bad → 400
✔ 5. GET /api/data/series?geography=../../etc/passwd → 400 (path traversal
rejected)
✔ 6. GET /api/data/quality → 200
✔ 7. GET /api/data/summary → 200
✔ 8. GET /api/neighborhoods/amenities → 200
✔ 9. GET /api/neighborhoods/amenities?amenity_type=school → 200
✔ 10. GET /api/data/series?start_date=2020-01-01&end_date=2019-01-01 → 400
✔ 11. GET /api/forecasting/predict?mode=demo → 200, status: ok, data_status:
synthetic_demo
✔ 12. GET /api/forecasting/predict?mode=demo&geography=mumbai-central&horizon=6 →
200
✔ 13. GET /api/forecasting/predict?mode=real → 200, status: unavailable (no real
data)
✔ 14. GET /api/forecasting/predict?horizon=99 → 400 INVALID_HORIZON
✔ 15. GET /api/forecasting/predict?mode=bad → 400 INVALID_MODE
✔ 16. GET /api/forecasting/predict?geography=../../etc → 400 INVALID_GEOGRAPHY
✔ 17. GET /api/forecasting/predict?metric=price_per_sqft → 400 INVALID_METRIC
```

7. Commands to Run

```
bash
cd "/Users/vanshbarnes/Documents/UrbanLens Project"

# Start Frontend (http://localhost:5173)
npm --prefix client run dev

# Start Backend API (http://localhost:3001)
npm --prefix server run dev

# Run Python Unit Tests
python3 -m unittest discover server/python/tests -v

# Run Server API Tests
npm --prefix server test

# Production Build
npm --prefix client run build
```

8. Limitations & Scope Boundaries

1. **Absence of Real Longitudinal Series**: Official RBI and NHB series have not yet been placed into `server/python/data/raw/`. The forecasting endpoint in `real` mode correctly reports `status: "unavailable"`.
2. **Lightweight Modeling**: Kept dependencies strictly to standard library and NumPy for M2 MacBook Air memory efficiency (no statsmodels or heavy neural network runtimes).
3. **Metropolitan Aggregation**: Real Indian housing price indices exist only at the city level, not the neighborhood level. The system explicitly distinguishes city indices from synthetic neighborhood capital rates.